Set → collection of properties.

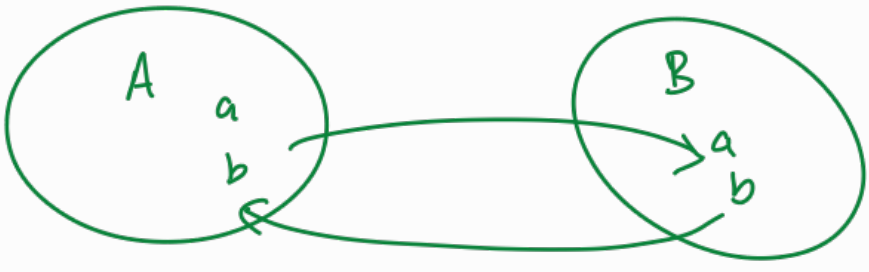
To express symbolically that an element belongs to a Set we use $\in$.

$A = \{1, 2, 3\}$

$1 \in A$, $2 \in A$

$4 \notin A$

Two Sets are equal if they contain the same element.



hence order doesn't matter

$A = \{1, 2, 3\}$ $B = \{2, 3, 3, 1, 2\}$

$A = B$

Doesn't matter if elements are repeated.

Size / cardinality of a Set is the no of elements it contains.

$A = \{1, 2, 3\}$ $|A| = 3$

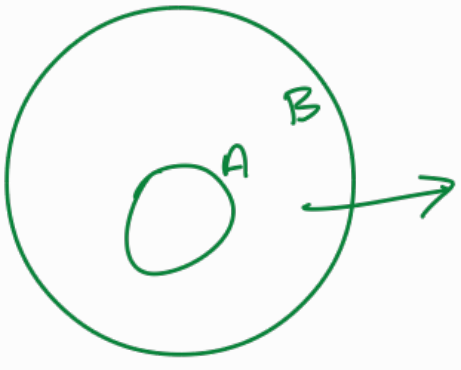
$A = \{2, 4, 6\}$ $B = \{1, 2, 3, 4, 5, 6\}$

$A \subseteq B$ → A is a subset of B

note: subsets are never a single element, because they are subsets

if $A \subseteq B$ and $B \subseteq A$ then $A = B$

if $A \subseteq B$ and $A \neq B$ then A is a proper subset of B.



elements in B but not in A.

Null :- $\{\Lambda\}$ 1 element exist of string with length 0.

emptyset :- $\{\}$ No element exist

Informal language → concerned with meaning

↳ english etc

work ✓ wrok (x)

Formal language (Automata focusses on formal languages)

concerned with Syntax

e.g starts with w and ends with 0.

work (x) wrk0 ✓

Language → is a set of strings made from Symbols. (Alphabets)

→ Alphabet is a small finite set of Symbols represented by (Σ) from which we build strings. $\Sigma = \{a, b\}$

2 Alphabets

Σ^* (Kleene Star) (all possible combinations including null)

Σ^+ (without including null)

$\Sigma^* = \{\Lambda, a, b, aa, ab, bb, ba, aaa\}$

→ Shorter strings come first

→ strings of same length are listed alphabetically

$$\Sigma^* - \{\Lambda\} = \Sigma^+$$

Making our own Language :-

$$\Sigma = \{a, b\}$$

$$\Sigma^* = \{\wedge, a, b, aa, ab, bb, ba, aaa, aab, bbb, bba$$

How did we combine them

$a + a \rightarrow aa$

$a + b \rightarrow ab$

$b + a \rightarrow ba$

$b + b \rightarrow bb$

Now (3-Letter words)

$aa + a$	$aa + b$
$ab + a$	$ab + b$
$ba + a$	$ba + b$
$bb + a$	$bb + b$

Possible combination

$2^n \rightarrow$ n is the length of the string

$2^3 \rightarrow 8$

$$\Sigma\{a, b\} = \{\wedge, a, b, aa, ab, bb, ba, aaa, aba, bba, baa, aab, abb, bbb, bab \ldots\ldots$$

$L_1 = \{x \mid x \in \Sigma^* \ \& \ x \text{ starts with } a\}$

$L_1 = \{a, aa, ab, aaa, aab, aba \ldots\}$

$L_2 = \{x \mid x \in \Sigma^* \ \& \ x \text{ ends with } ab\}$

$L_2 = \{ab, aab, bab \ldots\}$

$L_3 = \{x \mid x \in \Sigma^* \ \& \ x \text{ is a palindrome}\}$

$= \{\wedge, a, b, aba, aa, bb, aaa \ldots\}$ ↓ left ↔ right reading same

$L_4 = \{x \mid x \in \Sigma^* \ \& \ |x| \text{ is even}\}$ $|x|$ length of words is even.

$L_4 = \{\wedge, aa, bb, ab, ba, abab \ldots\}$

$L_5 = \{x \mid x \in \Sigma^* \ \& \ |x| \text{ is less than } 3\}$

$L_5 = \{\wedge, a, b, ab, ba, aa, bb\}$

$L_6 = \{x \mid x \in \Sigma^* \ \&$ any a is x (if any) should be followed by atleast $2b$s$\}$

$L_6 = \{\wedge, b, bb, bbb, bbbb, abb\}$

$\Sigma = \{a, b\}$

$\Sigma^* = \{\wedge, a, b, aa, bb, ab, ba, aaa, bba, aba, baa, aab, bbb, abb, bab\}$

$L_1 = \{x \mid x \in \Sigma^* \ \& \ x = a^i$ where i is a prime$\}$

$\{2, 3, 7, 11\}$

$aa, aaa, aaaaa, a^7$

$L_2 = \{x \mid x \in \Sigma^* \ \& \ n_a(x) = n_b(x)\}$

$\{\wedge, ab, ba, bbaa, aabb, baba\}$

$L_3 = \{x \mid x \in \Sigma^* \ \& \ x = a\}$

$\{a\}$

$L_4 = \{x \mid x \in \Sigma^* \ \& \ x = a \text{ OR } x = b\}$

$\{a, b\}$

$L_5 = \{ x \mid x \in \Sigma^* \ \& \ x = ab \}$

$\{ ab \}$

$L_6 = \{ x \mid x \in \Sigma^* \ \& \ x \text{ starts with } ab \}$

$\{ ab, aba, abb, \ldots \}$

$L_7 = \{ x \mid x \in \Sigma^* \ \& \ x = a^* \}$ → all possible combinations including null

$\{ \wedge, a, aa, a^3, a^4 \}$

general syntax

$L = \{ x \in \Sigma^* \mid \text{condition on } x \}$

↳ is a subset of Σ^*

Finite automata → means Self controlled machine

→ It is another method for defining languages.

→ It is a graphical method

e.g in ludo we change position on a input of certain no by dice until we reach the final state

→ Online input [input comes one by one). we even don't know whether the next input will

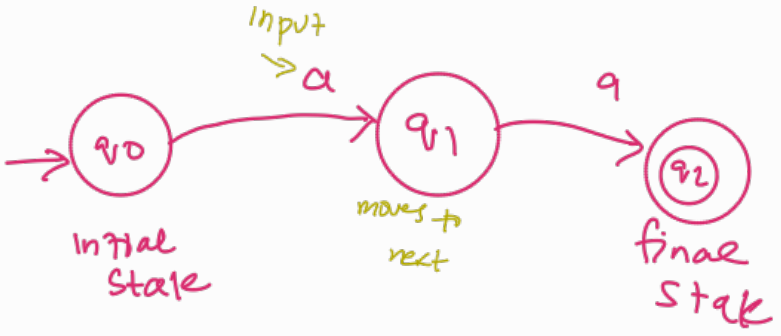
come or not.

→ offline input (we get the complete input once).

Q Define using finite automata.

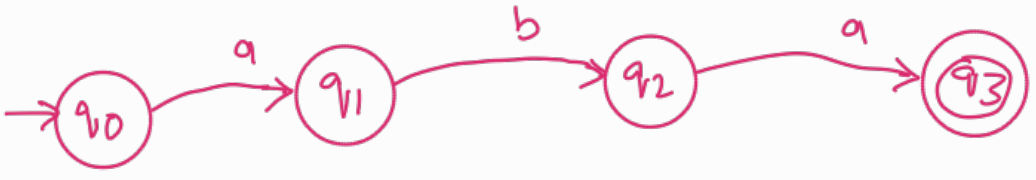
$L_1 = \{aa\}$

Note: The first a was already used when moving from $q_0 \to q_1$. it cannot be reused. At q_1 now the machine is waiting for the next symbol.



Accepts only aa.

$L_2 = \{aba\}$



Types of FA

(DFA)

only one output

(NDFA)

(we don't know any output)

(DFA)

FA → (Q, Σ, q_0, A, P)

Q → set of states — A, B, C

Σ → input — a, b

q_0 → initial state — A

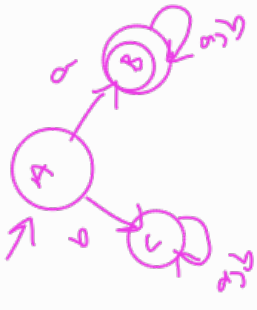
A → set of Accepting states — B

P → Transition function.

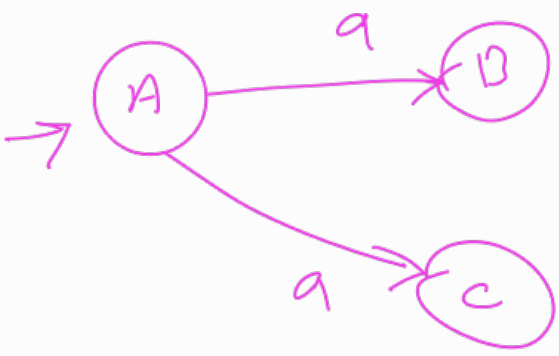
$Q \times \Sigma \to Q$

$(A, a) \to B$

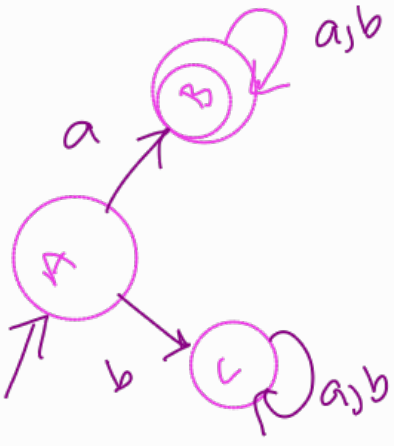
$(A, b) \to C$



non- deterministic this is not dfA



(With same input we go to two diff states)



$(A, a) \rightarrow B$
$(A, b) \rightarrow C$
$(B, a) \rightarrow B$
$(B, b) \rightarrow B$
$(C, a) \rightarrow C$
$(C, b) \rightarrow C$

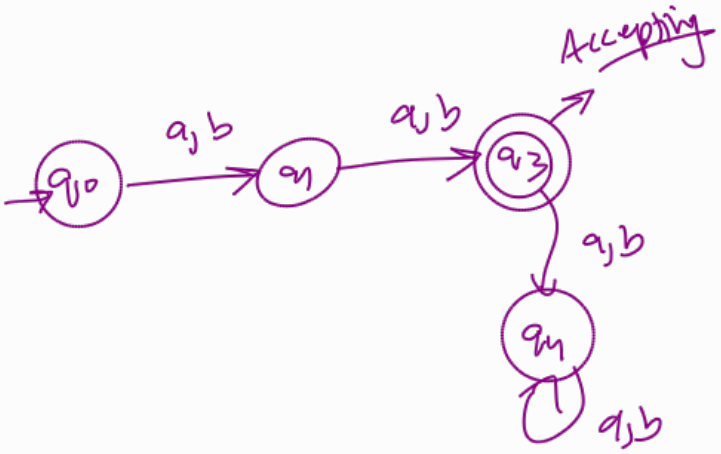
Total transitions

$\delta = Q \times \Sigma$
$= 3 \times 2$
$= 6$

In dfa we states must have a transition.

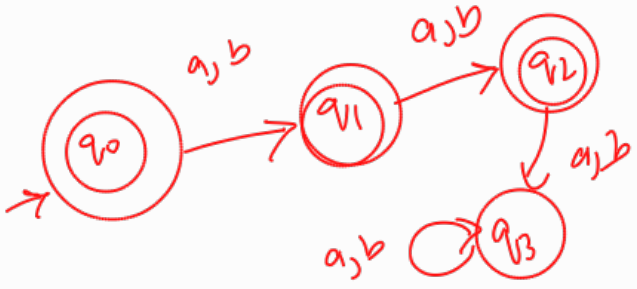
String that exactly contains

$\Sigma = \{a, b\}$
$|w| = 2$
aa, ab, ba, bb



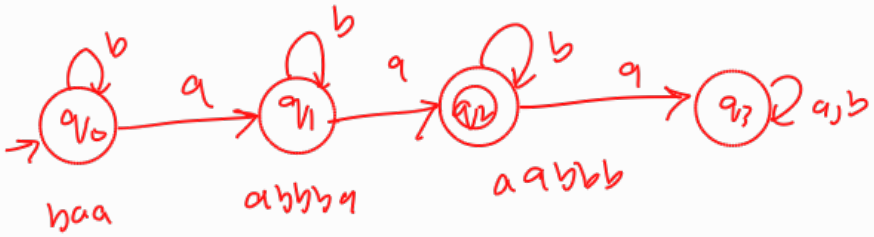
$|w| \leq 2$

$\Lambda, a, b, aa, bb, ab, ba$



i) $L = \{a, b\}$ $n_a(w) = 2$ $L = \{aa, baa, aba, aab, bbaa \cdots$

first draw min acceptance



mod 2 = 0
|| (even)
1 (odd)

ii) $n_a(w) \bmod 2 = 0$

↓

no of a's is even

aab ✓ 2 mod 2 = 0 ✓

∧ ✓ 0 mod 2 = 0 ✓

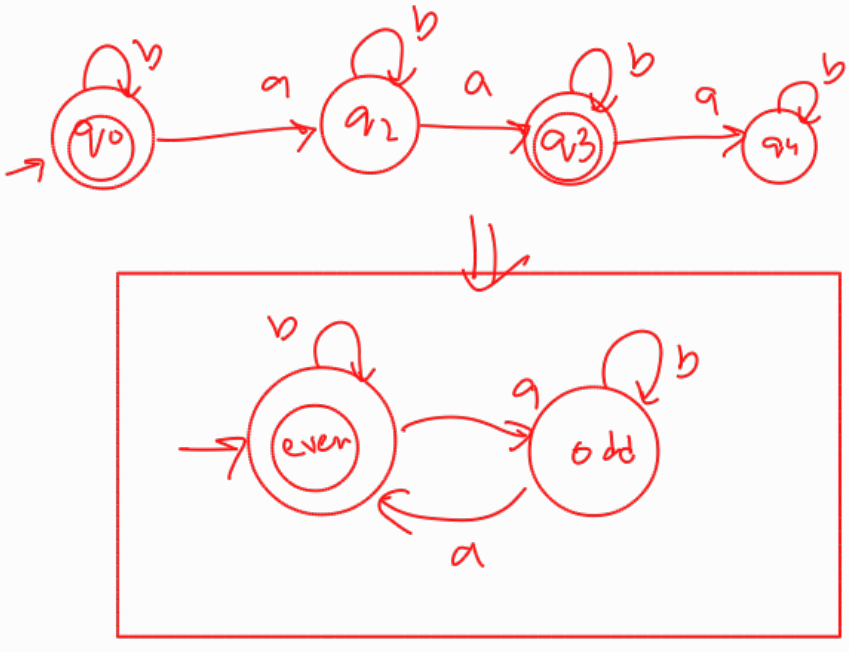
aaab (x)

a (x)

$L = \{a, b, aa, ab, ba, bb, aaa, aba, baa,$

$bba, aab, abb,$

$bab, bbb\}$



abaabaa

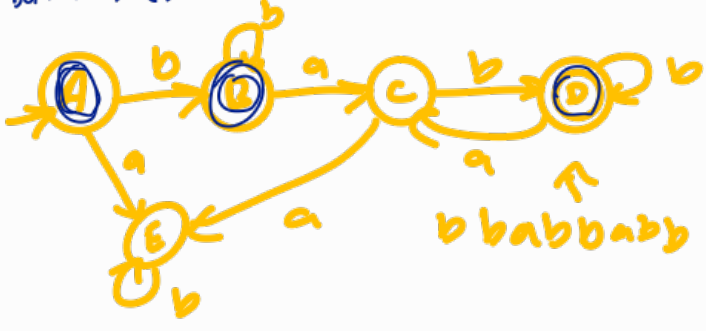
Design dfa $L = \{a, b\}$ that for every a in w is followed and preceded by b.

baba, ba(x), bababa (✓)

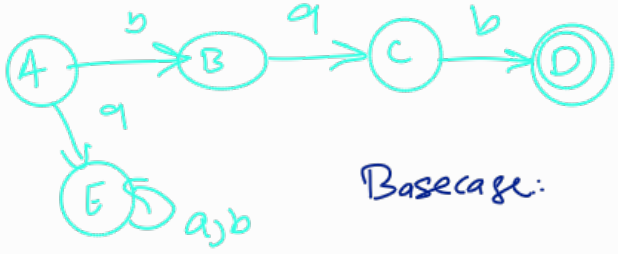
(✓)

this Q only accepts a followed and preceded by b and also b's, ∧.

But not ba(x)



bbbabbab



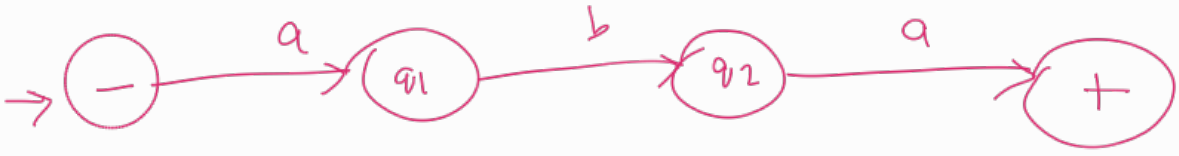
Basecase:

DFA of $L = \{aa\}$

in dfa we know at which state 'a' / input will go.



$L = \{aba\}$

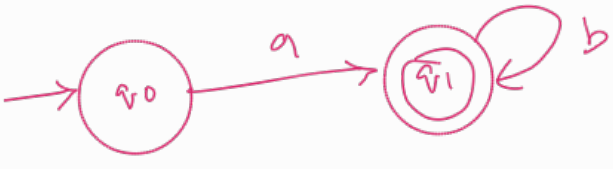


$L = a^*$



Starts with a and contains any number of b's in end.

$L = ab^*$



note:
State is just like a memory used to remember things

$q_0 \rightarrow$ start / nothing has been remembered yet.

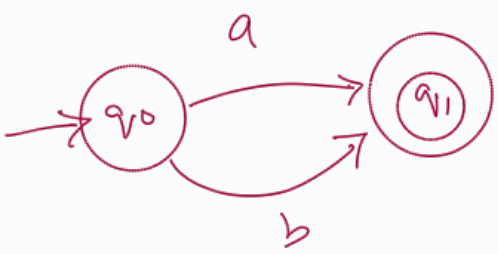
if we need to remember new things we need a new state

$q_1 \rightarrow$ a is seen hence move to q_1 and now accept any no of b's.

we don't need q_2 as the input b's are infinite

move to a new state if you need to remember to a specific condition.

Contains a or b.

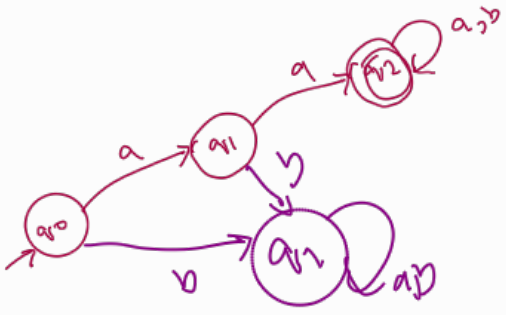


or

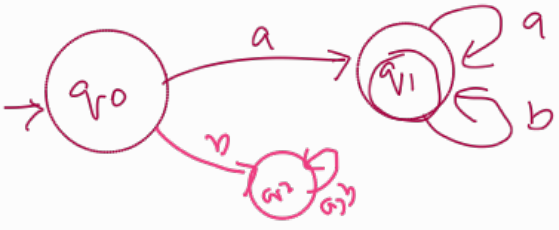


contains any no. of a's or b's





Starts with a.



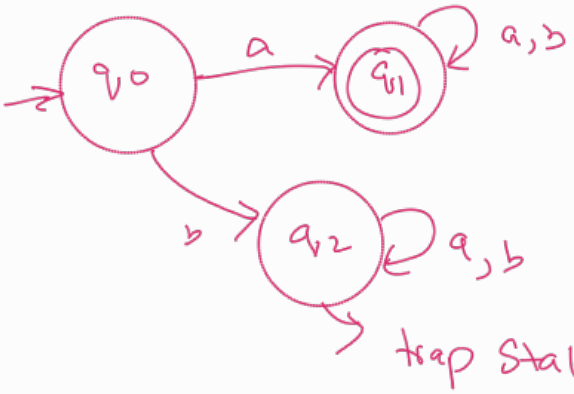
Starts with aa



$L = \{a, b\}$

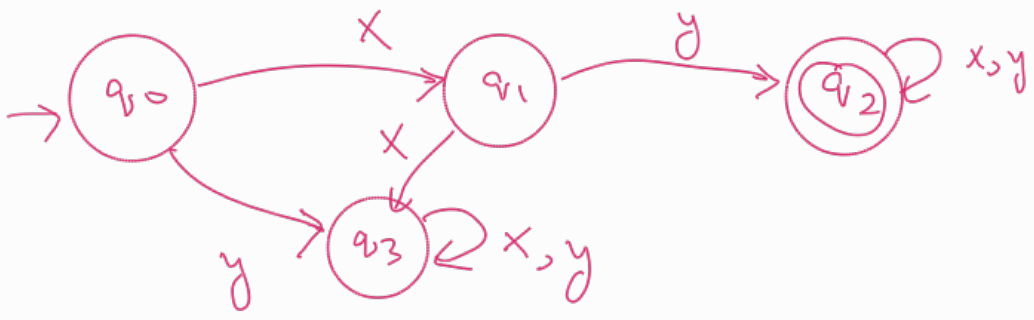
Starts with a

Rough

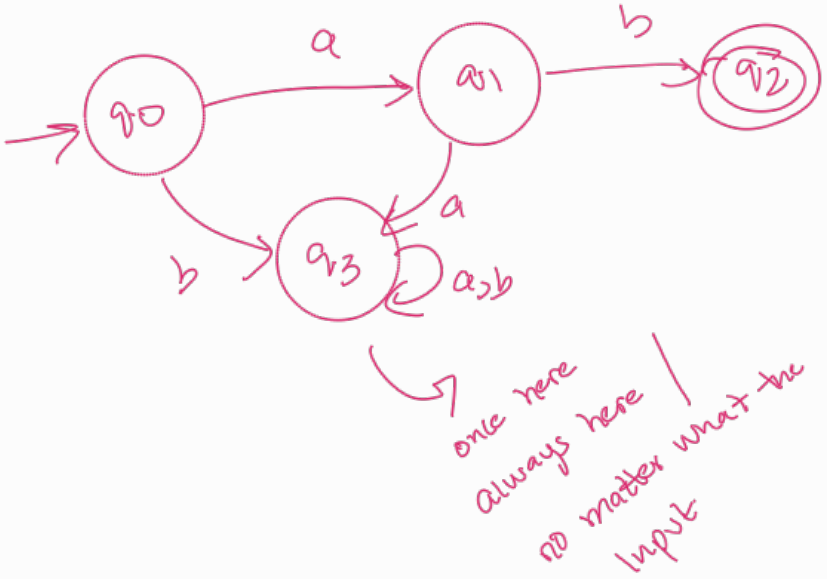


trap state → if b comes first. we will not reach the final state

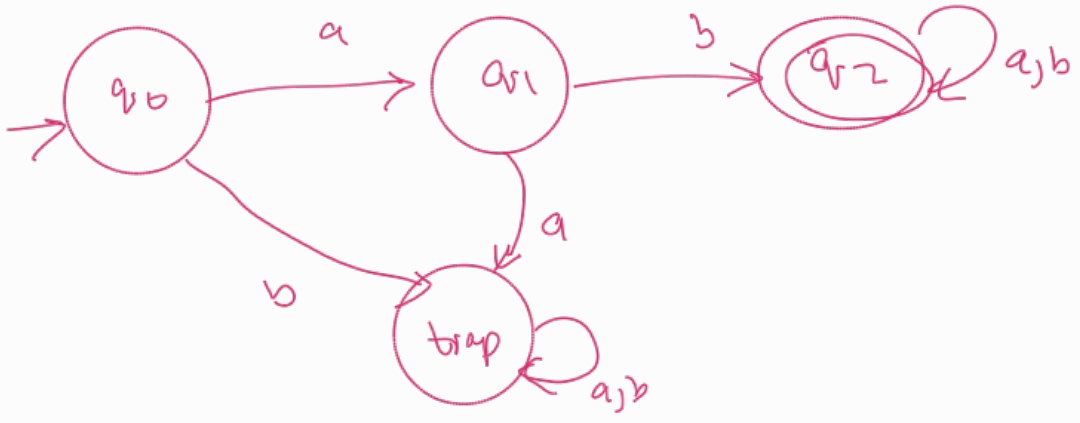
Construct an DFA which Recognizes set of all strings defined over $S = \{x, y\}$ with prefix $\langle xy \rangle$.



$\langle ab \rangle$

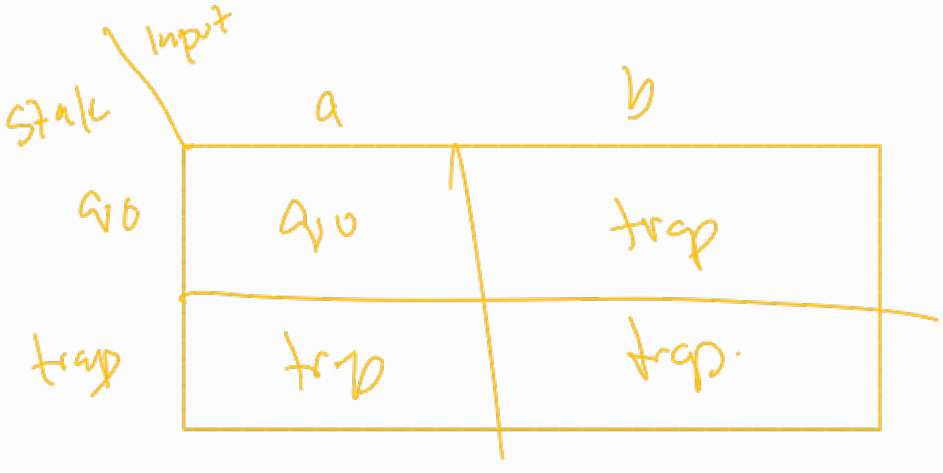


L_4 = Starts with ab.



L5 a^*



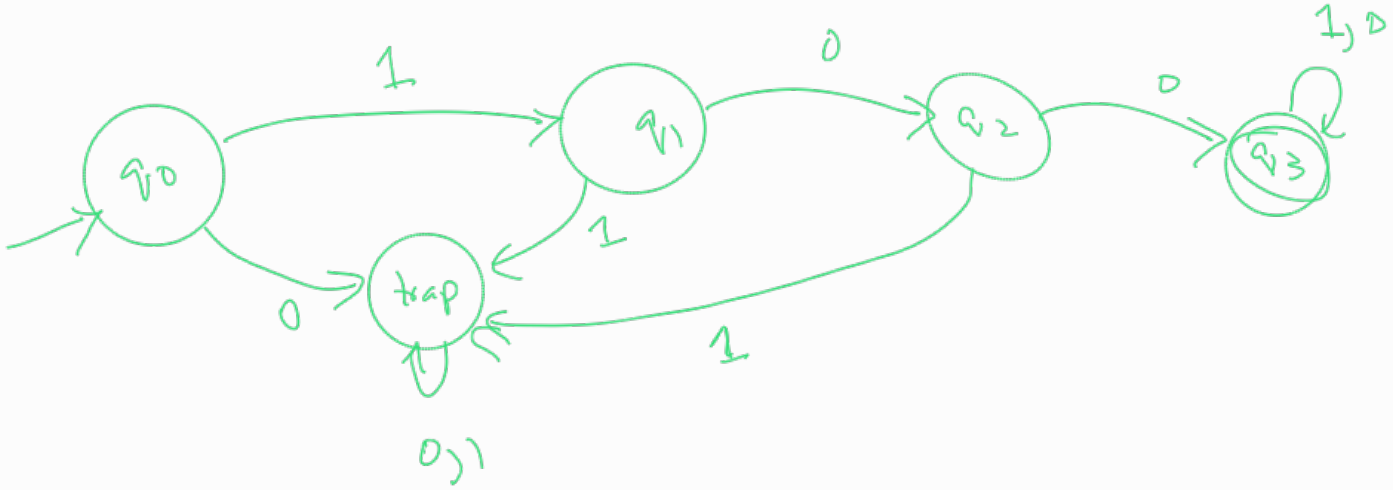


(q_0 → we know nothing yet)

$\Sigma = \{0,1\} \rightarrow (\wedge, 0, 1, 00, 01, 11, 10, 000, 001, 111, 110 \cdots)$

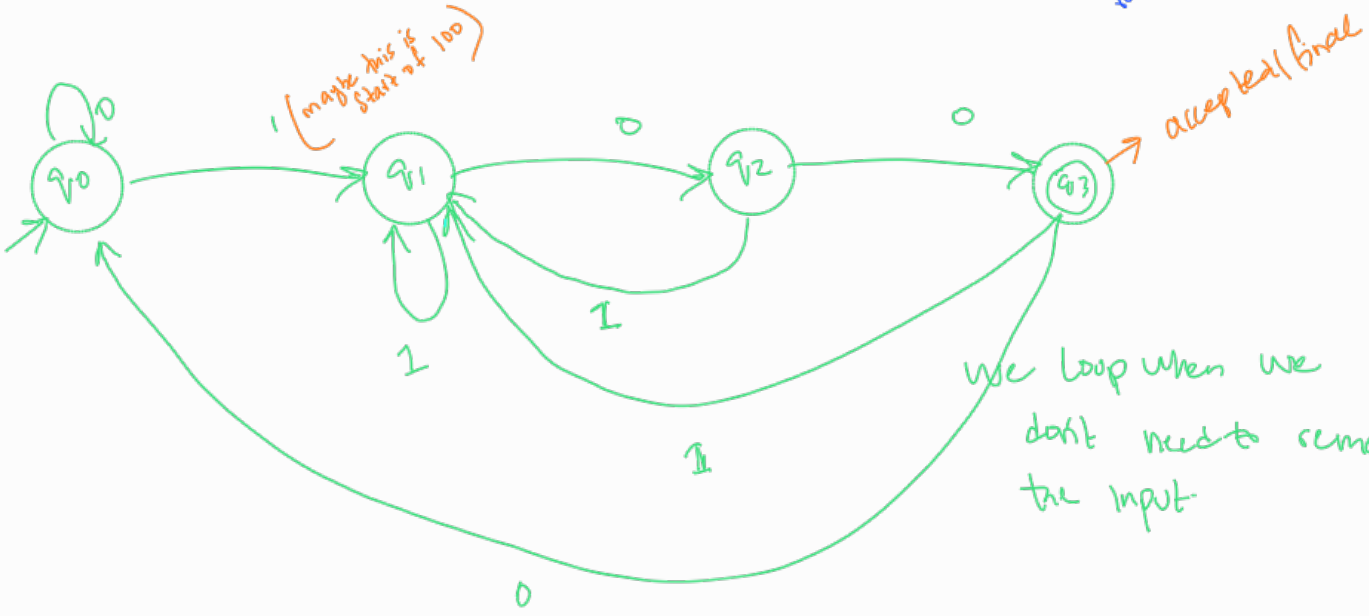
$L_1 = \{x \mid x \in \Sigma^* \text{ & } x \text{ starts with } 100\}$

Make DFA for this:



$L_2 = \{ x \mid x \in \Sigma^* \text{ s.t. } x \text{ ends with } 100 \}$

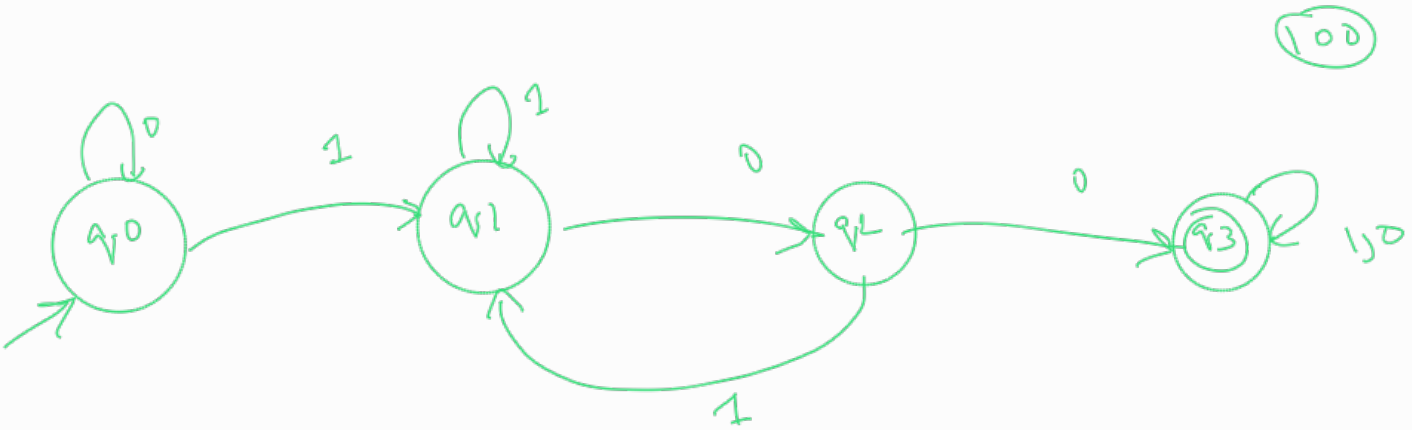
DFA needs to remember only the last 100.



Using trap will break the pattern and cannot cover all of the strings.

$L_3 = \{ x \mid x \in \Sigma^* \text{ s.t. } x \text{ has substring } 100 \}$ → can appear anywhere in the string

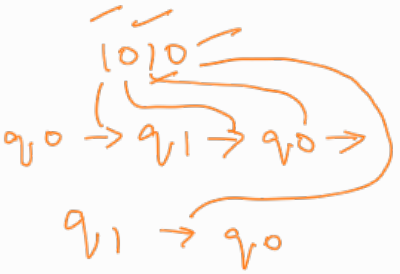
$\{100, 0100, 1100, 1000, 01001 \cdots\}$



$L_4 = \{ x \mid x \in \Sigma^* \text{ s.t. } |x| \text{ is even} \}$

$\{\Lambda, 00, 01, 11, 10 \cdots\}$

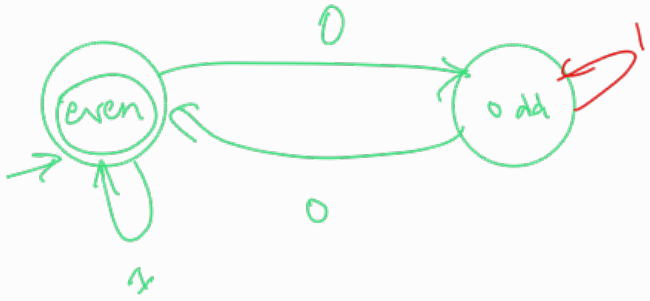




$L_5 = \{x \mid x \in \Sigma^* \text{ & } x \text{ has even } 0\text{'s}\}$

$(\Lambda, 00, 0000, 11, 111, \ldots)$

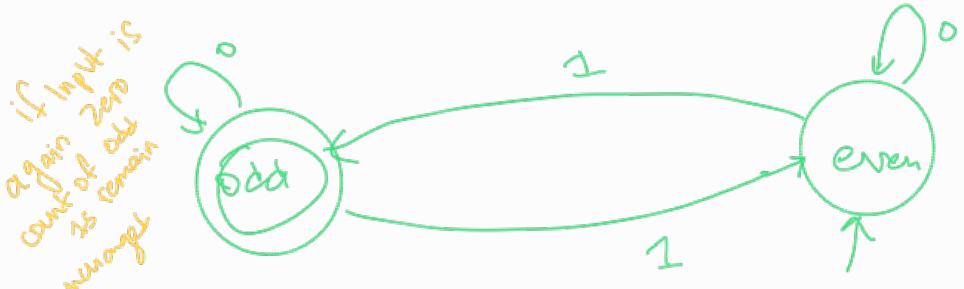
↘ doesn't matter how many ones.



00

(count of 0s doesn't change if input is 1)

$L_6 = \{x \mid x \in \Sigma^* \text{ & } x \text{ has odd } 1\text{'s}\}$



$\Lambda, 0, 1, 00, 01, 11, 10 \ldots$
× × ✓ ✓

control zero doesn't matter

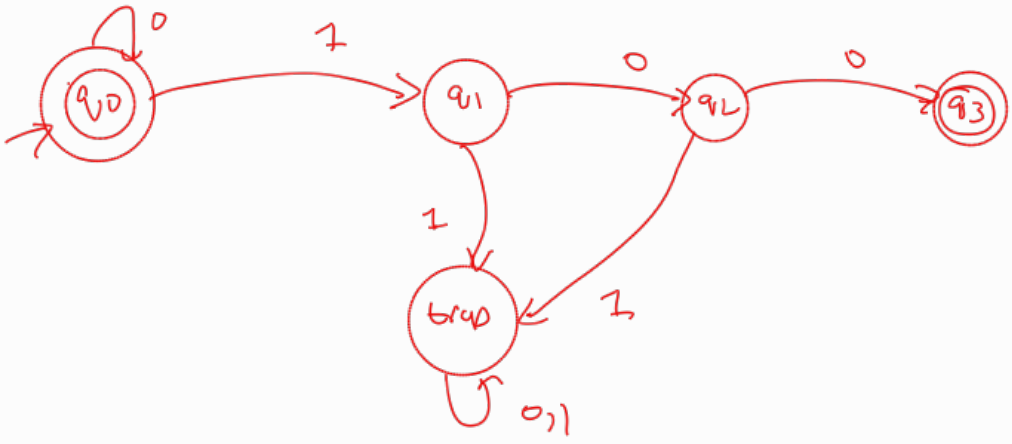
final state: Initial state only if null

$\Sigma = \{0, 1\}$ $\quad \wedge, 0, 1, 00, 01, 11, 10, 000, 010, 110, 100 \ldots$

$L_1 =$ $\Sigma\, x | x \in \Sigma^*$ & every 1 in x (if any) should be followed by atleast two 0s$\}$

$\wedge, 0, 00, 100$

100



$1^* \rightarrow \wedge, 1, 1^2, 1^3, 1^4$ $\quad \wedge = "n$

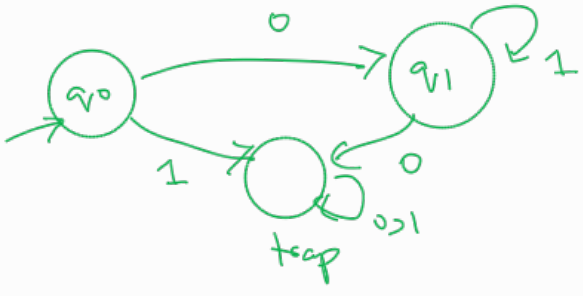
$01^* \rightarrow 0\wedge, 01, 011, 011, 0111 \ldots$

$\downarrow$

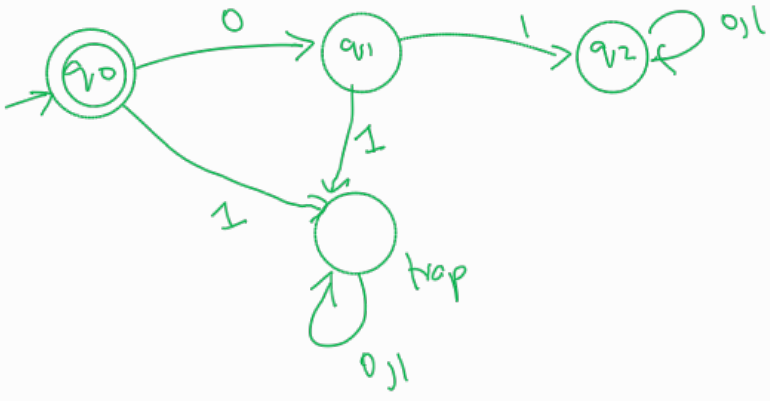
0

$(01)^* \rightarrow \wedge, 01, 0101, 010101 \ldots$

$L_8 = \{x | x \in \Sigma^* \text{ \& } x = 01^*\}$



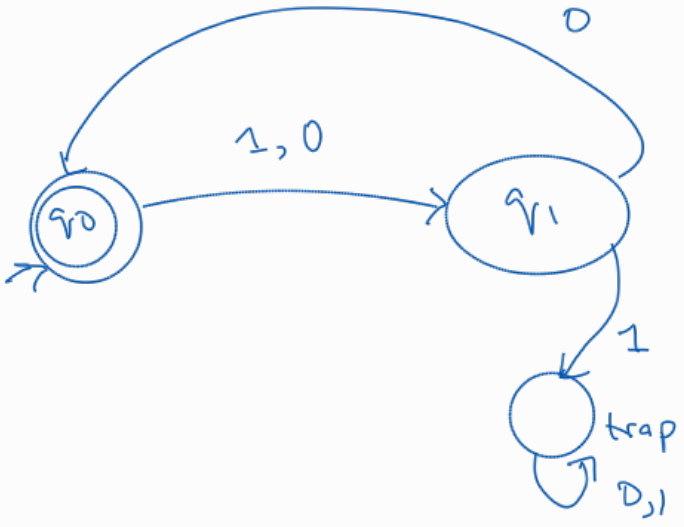
$L_9 = \{x | x \in \Sigma^* \text{ \& } x = (01)^*\}$



L_{10} = $\{x / x \in \Sigma^*$ & every second element (if any) should be 0$\}$

$\wedge, 0, 1, 00, 01, 11, 10, 000, 010, 110, 100 \ldots$

$\wedge, 0, 1, 00, 10, 000, 100 \ldots$



$\Sigma = \{0, 1\}$

L_1 = $\{x / x \in \Sigma^*$ & $n_0(x)$ is even & $n_1(x)$ is odd $\}$

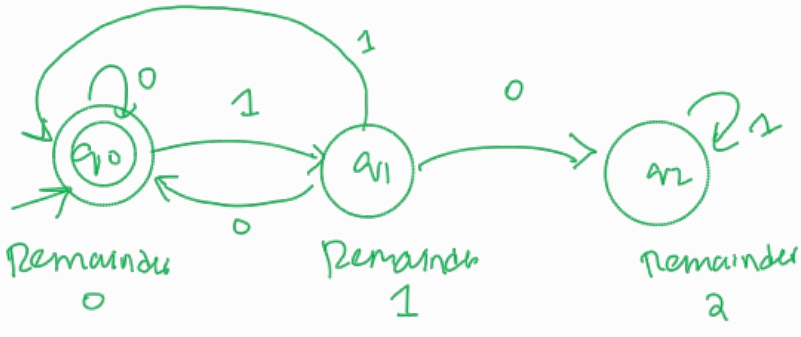
n_0 = no of 0's in x

n_1 = no of 1's in x

$1, 000, 010, 110 \ldots$

L_2 = $\{x / x \in \Sigma^*$ & binary of x is divisible by 3$\}$

Remainder after dividing any no with 3 → 0, 1, 2 ← no. of states



$(01)^*$ → repeat this again

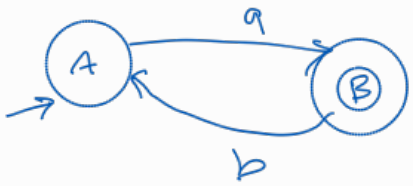
$L_4 = \{x \mid x \in \Sigma^*$ where x has atleast three a's and all strings containing no more than one occurence of the substring bb.

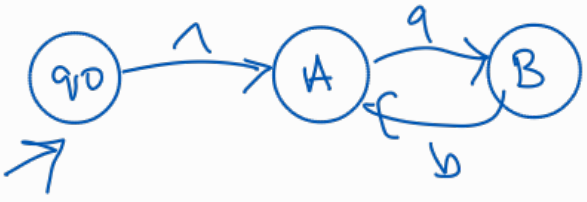
$L = \{a, b\}$

$\Sigma = \wedge, a, b, aa, ab, ba, bb \ldots$

State elimination is used to convert DFA to Regular expressions.

1)

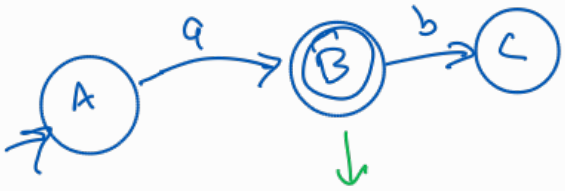


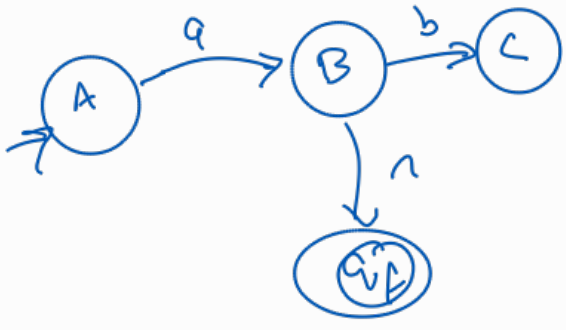


if intial state has any incomming edge

then we will creak a new intial state with no Incomming edge.

2)

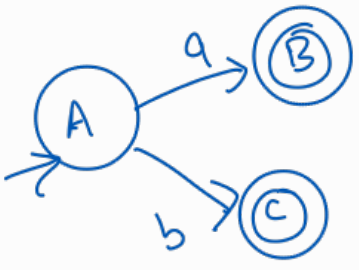


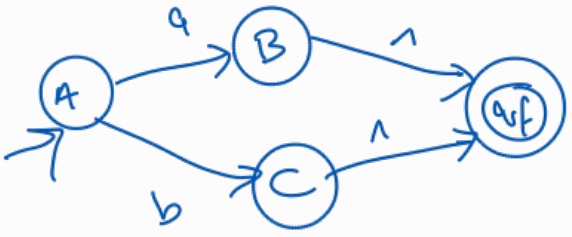


if final state has any outgoing state

Then we will creak a new final state.

3)



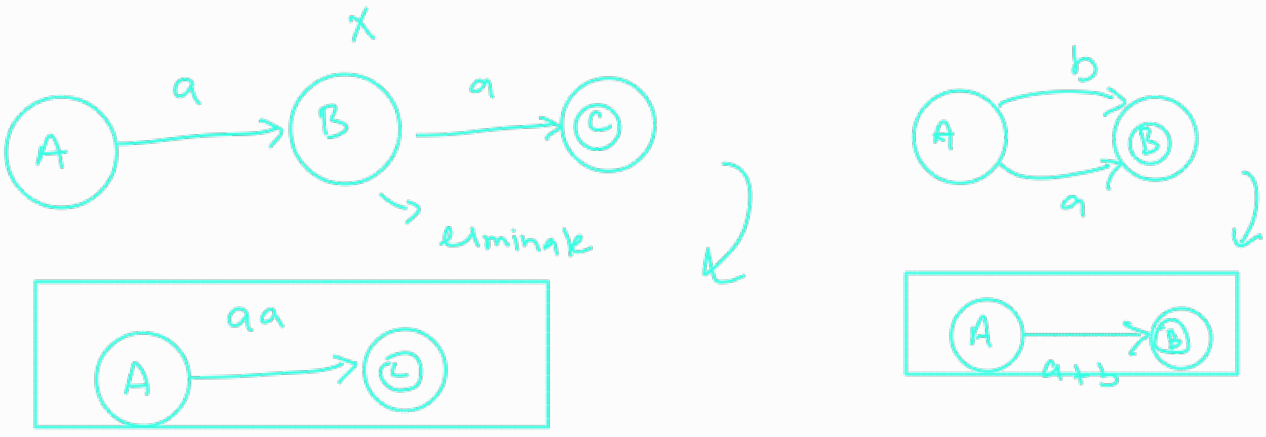


If there are more than 2 final states, then convert it to a single final state.

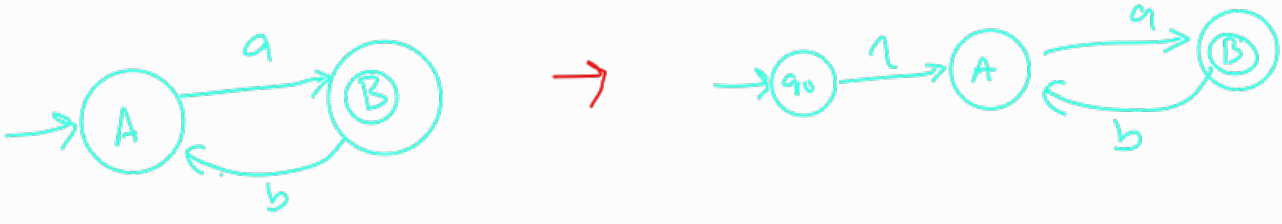
Self loop is both incomming | outgoing

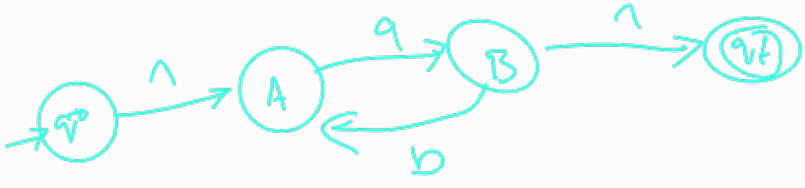
4) Eliminate all States one by one Except Initial | final State

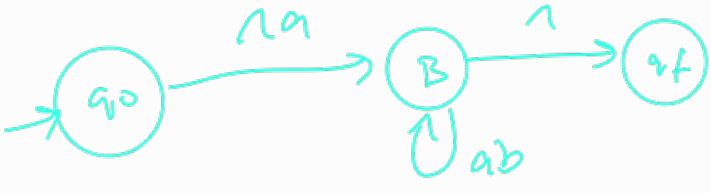
e.g

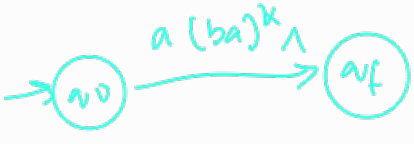


note: Apply these Rules in order

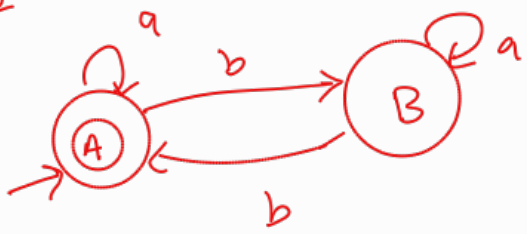


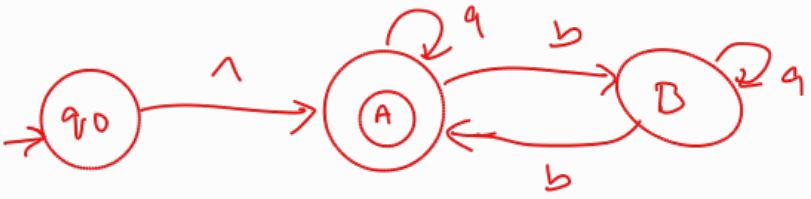


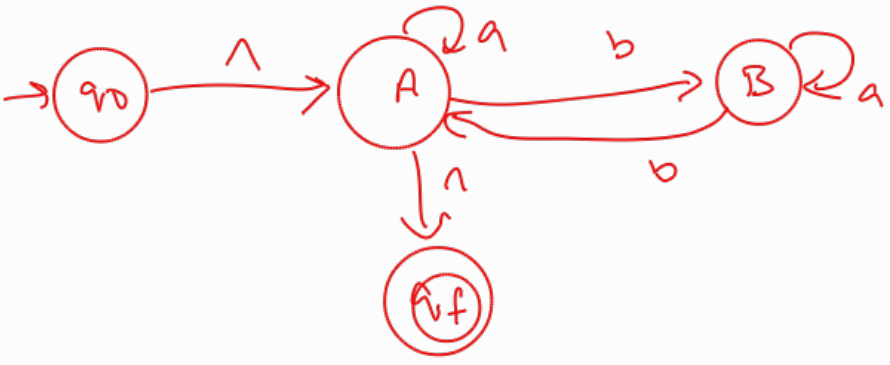




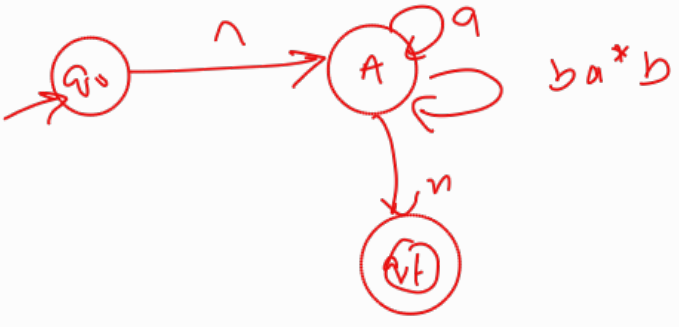
eg2



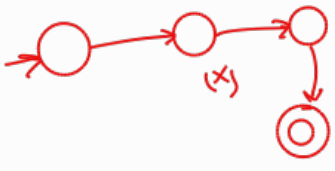




ba^*b



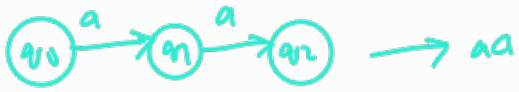
It is easier to eliminate that state that is not next to final state



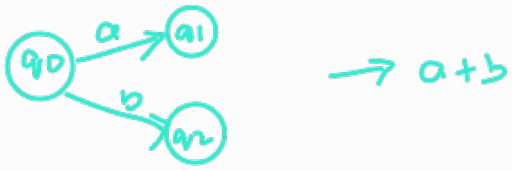


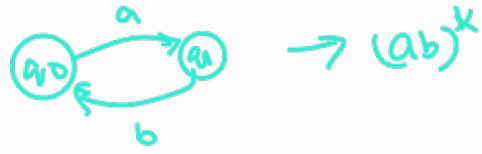
$\wedge (a + ba^*b)^* \wedge$

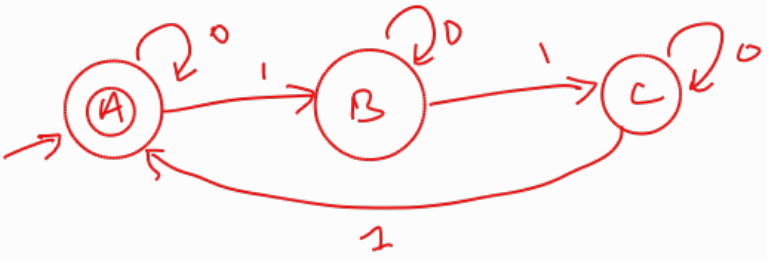

q0
a
a*

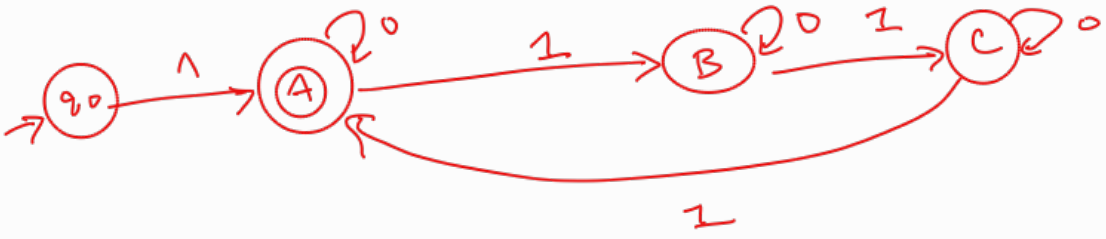

q0
a
q1
a
q2
aa

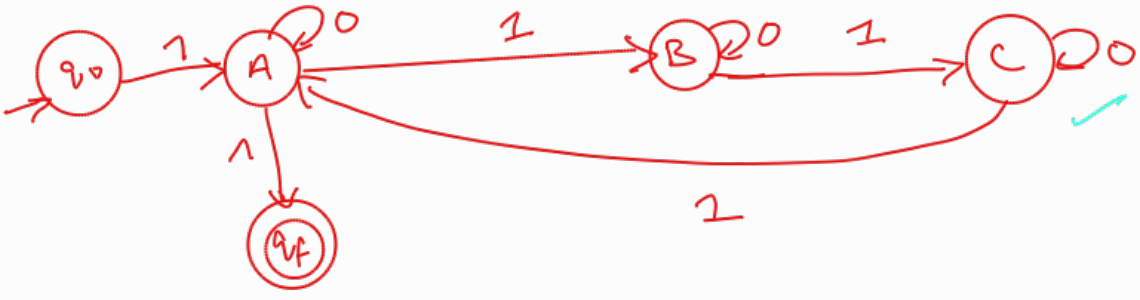

q0
a,b
(a+b)*

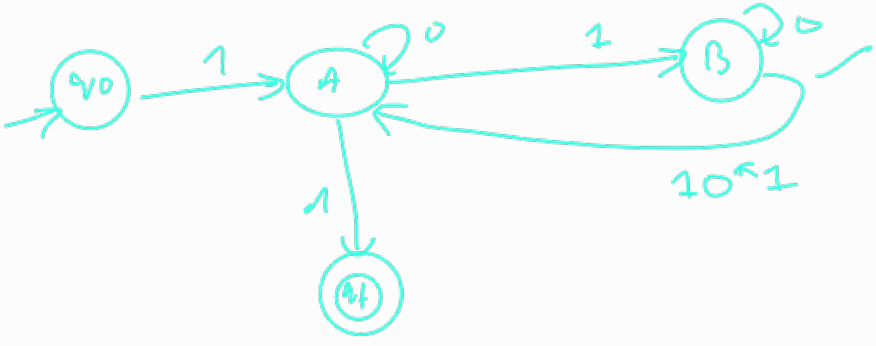

q0
a
q1
b
q2
a+b

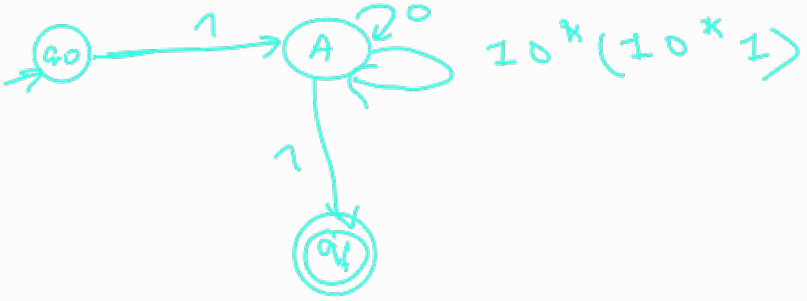

q0
a
q1
b
(ab)*

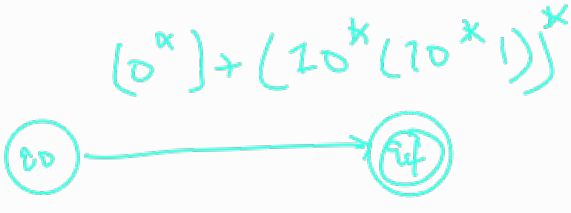

A
B
C
0
1
0
1
0
1


q0
1
A
0
1
B
0
1
C
0
1


q0
1
A
0
1
B
0
1
C
0
1
qf


q0
1
A
0
1
B
0
10*1
1
qf


q0
1
A
0
10*(10*1)
1
qf

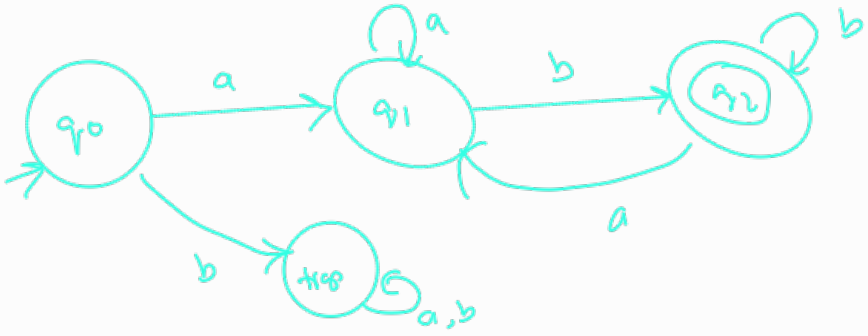

0*+(10*(10*1)*
q0
qf

(0+10*10*1)*

Construct a DFA which accept a Language of all Strings Starting with 'a' and ending with 'b'

$\Sigma = \{a, b\}$ $\{\wedge, a, b, aa, ab, bb, ba, aaa, aba, baa, bbb, abb, \}$

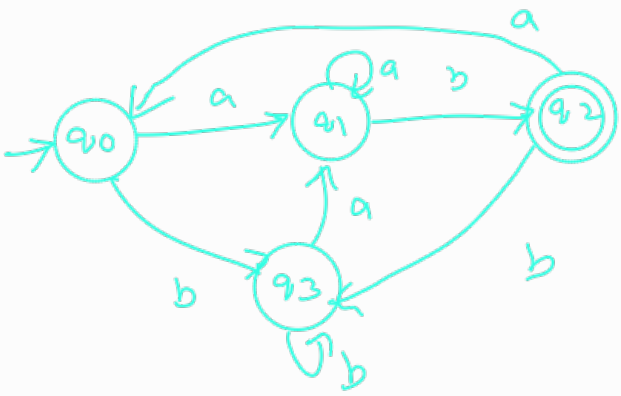
Design Basic DFA for min string that accepts then fill in the gaps. (ab)

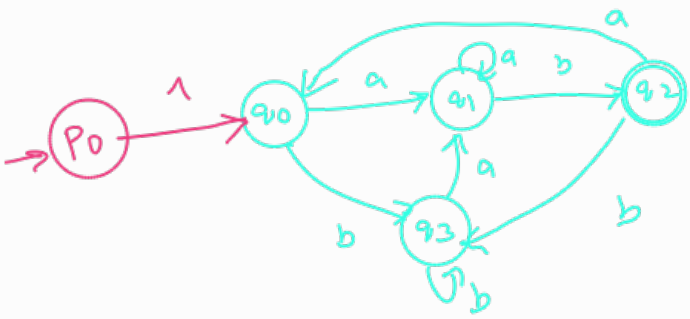


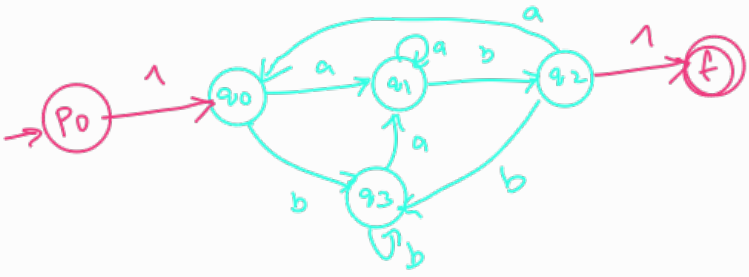
e.g construct a DFA of all strings not starting with a or not ending with b.

$L = \{\wedge, a, b, bb, aa, ba, aaa, bbb \ldots$

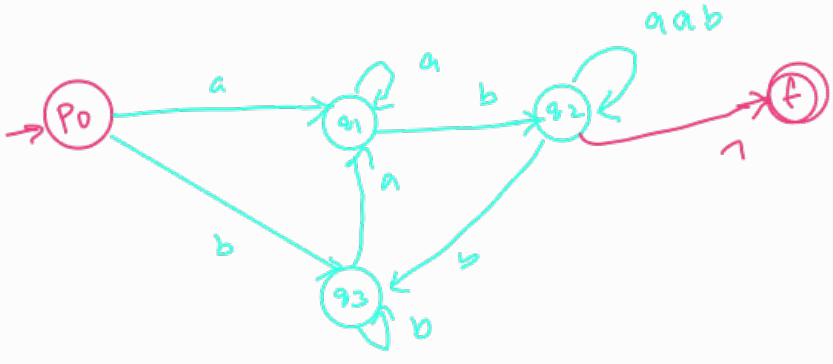
State elimination of this diagram:



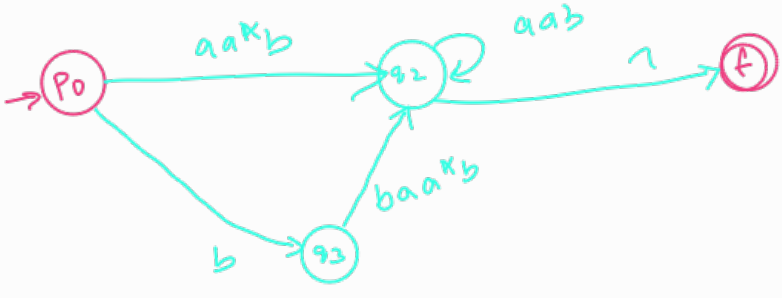


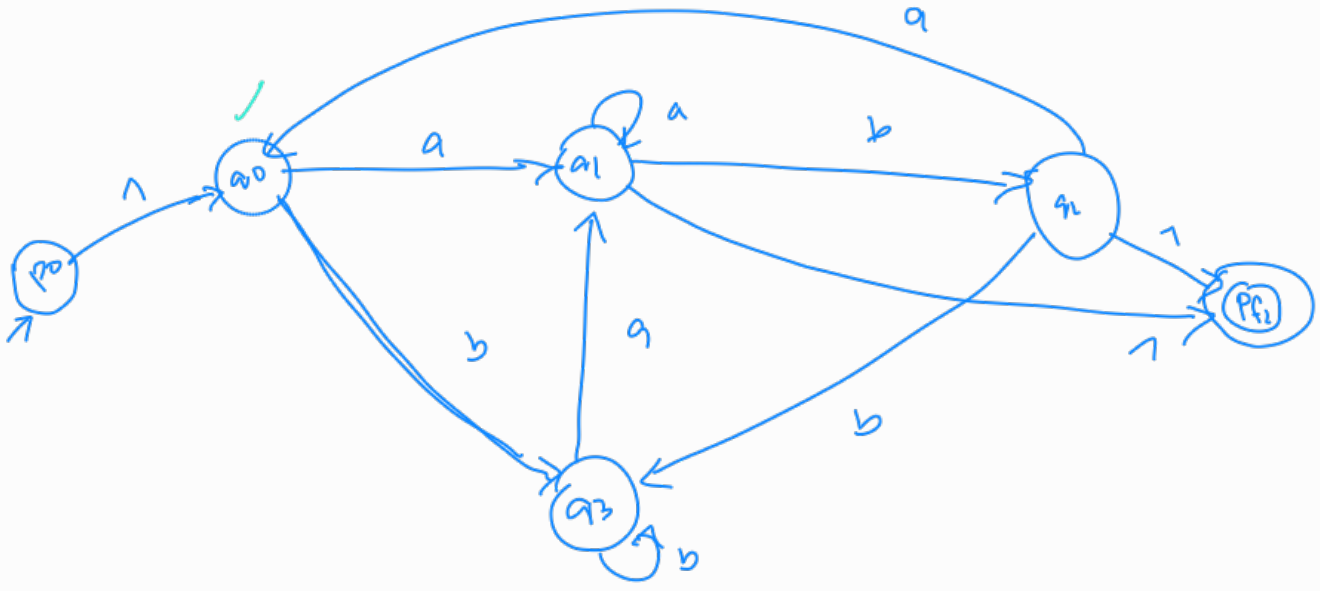


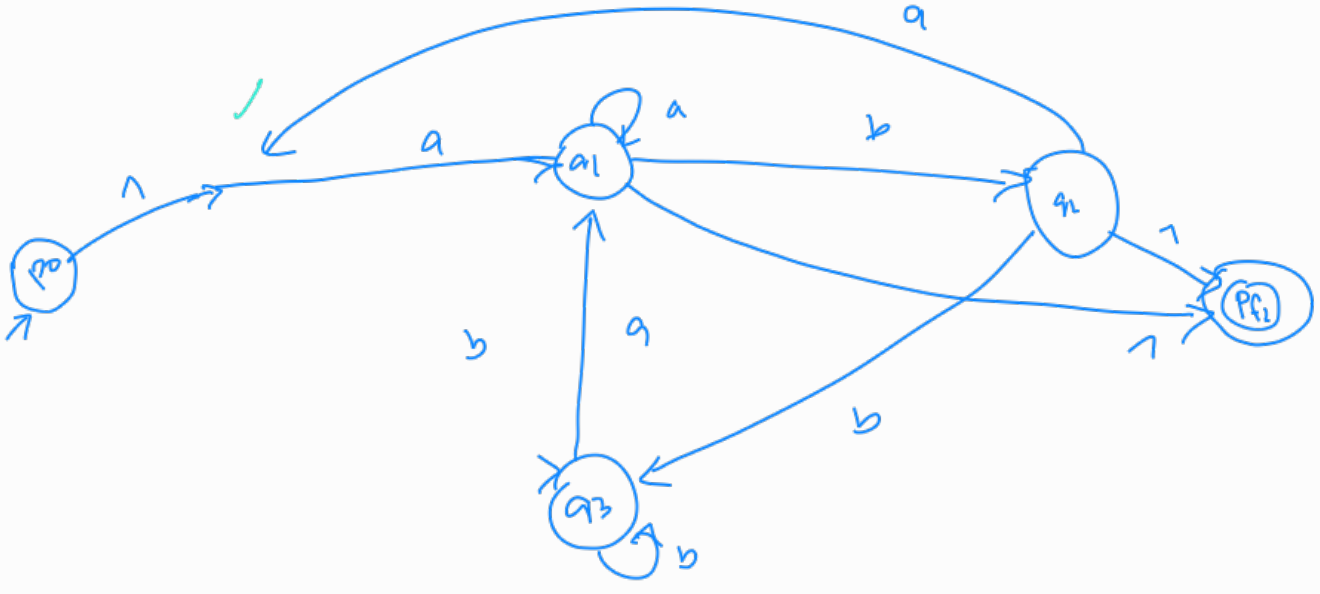
eliminating q_0



eliminating q_1




a
a
a
b
λ
λ
b
a
b


a
a
a
b
λ
λ
b
a
b

$L = a^3 b\, w a^3 \mid w$ is any string over $\Sigma a, b 3$.

aaab aba b aaa

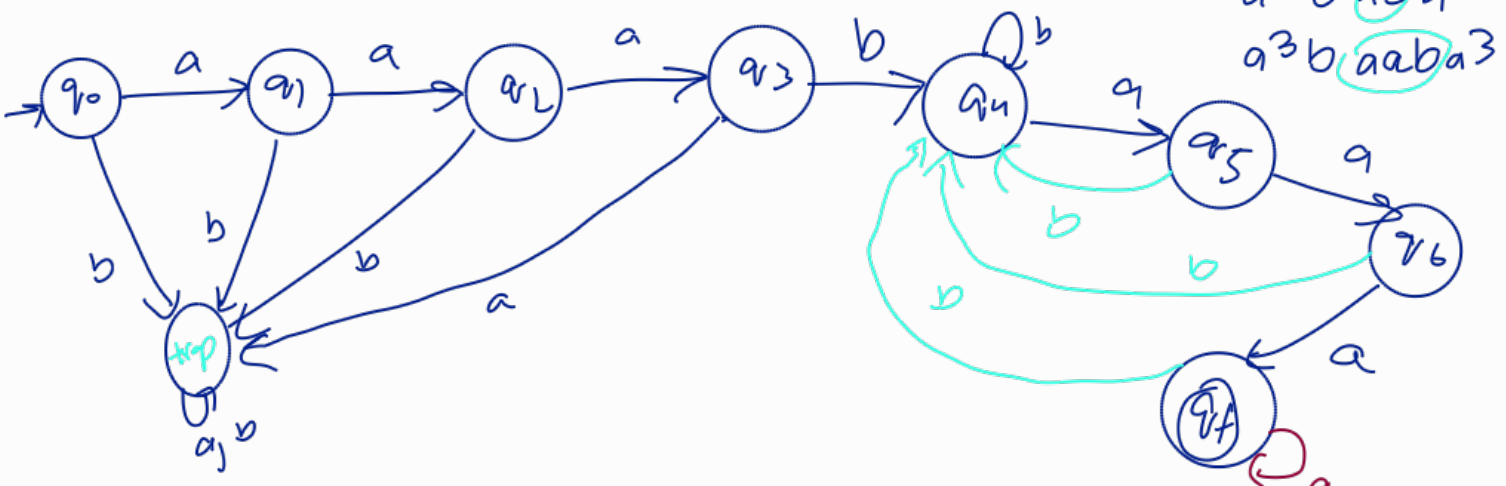
aaaba aaba ba aa aaa

Base case aaab ∧ aaa

$a^3 b bbb a^3$

$a^3 b\, ab\, a^3$

$a^3 b\, aaba^3$



$a^3 b aaaa$ aa

$a^3 b aaab aaa$